

¿Qué es Jest y React Testing Library? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Jest es un framework de pruebas en JavaScript desarrollado por Meta diseñado para ejecutar y evaluar código.

Características:

- **Motor de ejecución:** actúa como el entorno principal que busca los archivos de prueba, ejecuta las rutinas y muestra los resultados de éxito o fallo en la consola.
- **Aserciones:** provee de funciones clave como `expect()` para comparar los resultados esperados contra los obtenidos.
- **Simulaciones (mocking):** permite aislar componentes reemplazando dependencias externas, funciones complejas o llamadas a APIs con datos sintéticos.
- **Cobertura de código:** genera reportes sobre qué porcentaje del código fuente está siendo evaluado por las pruebas.

Básicamente es `jUnit` + `mockito` + `jacoco` todo en uno y para javascript.

Por otro lado, React Testing Library es una biblioteca de JavaScript diseñada para probar componentes de React desde la perspectiva del usuario final.

Características:

- **Pruebas basadas en el comportamiento:** simula acciones reales como hacer clic en un botón o escribir texto, en lugar de probar detalles internos del código o estados del componente.
- **Búsqueda semántica:** permite atrapar elementos visuales a través del texto visible, etiquetas o roles de accesibilidad (ej. `getByText`, `getByRole`).
- **Trabajo con el DOM real:** renderiza nodos del DOM real o simulado mediante herramientas como Jest o Vitest, facilitando consultas de elementos igual a cómo los ve un usuario.
- **Estándar de la industria:** Es la herramienta recomendada oficialmente por el equipo de React para el ecosistema de pruebas en el frontend.

Ejemplo utilizando ambos:

```
// Componente: Saludo.jsx
import { useState } from 'react';

export default function Saludo() {
  const [mensaje, setMensaje] = useState("");

  return (
    <div>
      <h1>¡Bienvenido!</h1>
      <button onClick={() => setMensaje("Hola")}>Saludar</button>
      {mensaje && <p>{mensaje}</p>}
    </div>
  );
}
```

```
// Prueba: Saludo.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';
import Saludo from './Saludo';

test('muestra el mensaje al hacer clic en el botón', () => {
  // renderizar con la función render de RTL
  render(<Saludo />);

  // aca usamos Jest junto con RTL. las funciones getBy dijimos que son
  // propias de RTL
  const titulo = screen.getByRole('heading', { name: /¡bienvenido!/i });
  expect(titulo).toBeInTheDocument();

  // RTL interactúa con la interfaz
  const boton = screen.getByText('Saludar');
  fireEvent.click(boton);

  // aca se valida que el mensaje correcto haya aparecido en pantalla
  const mensajeAparecido = screen.getByText('Hola');
  expect(mensajeAparecido).toBeInTheDocument();
});
```

Al usar `render(componente)` RTL lo dibuja internamente en un **DOM virtual aislado**.

`screen.getByRole()` y `screen.getByText()` son funciones de RTL que escanean la pantalla virtual buscando el título y el botón como lo haría un usuario viendo la página.

La función `getByRole` tiene un segundo parámetro (opcional) que se usa en caso de que hayan varios elementos con el mismo rol. En este caso el argumento es `name`, que funciona como filtro indicando qué texto visible o etiqueta debe tener el elemento para ser el que se busca. En este ejemplo indica que buscamos una etiqueta html de tipo `h` que tenga el texto ¡bienvenido!.

La otra función de RTL que aparece en el ejemplo es `fireEvent.click()`, que ejecuta un clic simulado sobre el botón capturado.

Las funciones `test(...)` y `expect(...)` son de Jest. Envuelven el proceso y auditan que el texto nuevo exista (`toBeInTheDocument()`) tras la interacción, validando finalmente si la prueba pasa o falla.